Fundamentos para el desarrollo de aplicaciones web

Breve descripción:

Este componente es una guía técnica y pedagógica para desarrollar competencias en maquetación, estructuración y despliegue de soluciones web. En diferentes unidades temáticas aborda fundamentos de arquitectura web, comunicación cliente-servidor (client-server), desarrollo frontend y backend, especificación de requerimientos técnicos y analítica de información, aplicando estándares de interoperabilidad y protocolos de red para la mejora continua del software.

Septiembre 2026

Tabla de contenido

Introducción 4

1. Arquitectura web 7

1.1. Conceptos y características 7

1.2. Tipos de arquitectura web 9

1.3. Mapas de navegación y procedimiento para diseñar 12

1.4. Técnicas de integración y sintaxis de contenidos web 13

1.5. Características y licenciamiento de la plataforma 16

2. Comunicación y diseño web 18

2.1. Técnicas de comunicación 18

2.2. Protocolos de comunicación 19

2.3. Características del diseño web 24

3. Desarrollo web 29

3.1. Técnicas de desarrollo web 29

3.2. Herramientas de desarrollo web 32

3.3. Características de las herramientas 34

4. Sitio web 37

4.1. Definición y características 37

4.2. Tipos y categorías 38

4.3. Características de los elementos 39

4.4. Requerimientos funcionales y no funcionales 40

4.5. Parámetros de configuración, descripción y palabras clave 41

5. Análisis de información 44

5.1. Concepto de análisis de información 44

5.2. Procedimientos de análisis 45

5.3. Modelos de resultados 47

5.4. Técnicas de evaluación 49

5.5. Conclusiones 50

Síntesis 53

Glosario 55

Referencias bibliográficas 57

Créditos 58

Introducción

En el panorama tecnológico contemporáneo, el desarrollo de aplicaciones y sitios web ha dejado de ser un proceso meramente estético para convertirse en una disciplina de ingeniería que exige un alto rigor técnico. La masificación de dispositivos con diversas capacidades de procesamiento y resoluciones de pantalla, sumada a la heterogeneidad de los sistemas operativos y navegadores (browsers), plantea el desafío de construir productos digitales que garanticen una experiencia uniforme, accesible y eficiente para cualquier usuario.

En este contexto, el componente formativo adquiere una relevancia estratégica dentro de la formación técnica, al abordar la construcción de sitios web mediante técnicas de interoperabilidad y protocolos técnicos. La interoperabilidad, entendida como la capacidad de diferentes sistemas informáticos para intercambiar datos de manera transparente y precisa, depende de la adopción de estándares internacionales establecidos por organismos como el World Wide Web Consortium (W3C) y la Internet Engineering Task Force (IETF).

El propósito central de este material es dotar a los aprendices de los conceptos, herramientas y metodologías fundamentales para abordar el ciclo de vida del desarrollo web. A lo largo del contenido, se examinan las arquitecturas monolíticas y desacopladas, los protocolos seguros de transferencia de datos, como HTTPS, las mejores prácticas en usabilidad (UX) y accesibilidad (WCAG), así como el uso de herramientas modernas de control de versiones y entornos de desarrollo. Finalmente, se destaca la importancia de evaluar sistemáticamente los resultados mediante

analítica de datos, promoviendo una cultura de optimización basada en evidencias e indicadores clave de rendimiento.

Para comprender la importancia del contenido y los temas abordados, se recomienda acceder al siguiente video:

Video 1. Fundamentos para el desarrollo de aplicaciones web



Enlace de reproducción del video

Transcripción del video. Fundamentos para el desarrollo de aplicaciones web
El desarrollo de soluciones web ha evolucionado desde páginas orientadas principalmente a mostrar información hasta aplicaciones capaces de conectar usuarios, servicios y datos en distintos dispositivos y entornos. Esta evolución exige comprender qué ocurre detrás de una interfaz y cómo sus diferentes componentes logran comunicarse.

Transcripción del video. Fundamentos para el desarrollo de aplicaciones web
Cada interacción depende de una arquitectura que organiza el sistema y de protocolos de comunicación que permiten intercambiar información entre clientes y servidores. A esto se suman criterios de diseño responsivo y accesibilidad, necesarios para que los contenidos puedan adaptarse a diferentes pantallas y condiciones de uso. La construcción de estas soluciones también requiere integrar tecnologías de frontend y bachead. El frontend estructura y presenta la interfaz, mientras el backend procesa solicitudes, aplica la lógica de la aplicación y gestiona los datos. Su funcionamiento se complementa con herramientas de desarrollo, control de versiones y configuración. Estos aspectos forman parte de los ejes desarrollados en el componente. A lo largo del componente, estos fundamentos se abordarán mediante explicaciones técnicas, ejemplos de código y situaciones aplicadas, que permitirán analizar cómo se estructuran, configuran y comunican los elementos de una solución web. Finalmente, el análisis de datos y métricas permitirá relacionar el desarrollo con la evaluación de resultados, identificando oportunidades para optimizar el rendimiento y fortalecer la mejora continua de sitios y aplicaciones web.

1. Arquitectura web

La arquitectura web constituye el conjunto de patrones estructurales, principios de diseño y modelos de abstracción que rigen la interacción entre los componentes de software en una red distribuida. No se limita únicamente a la maquetación o disposición estética del contenido; comprende el diseño conceptual de la infraestructura lógica, la gestión eficiente del flujo de datos, la distribución de servicios y la optimización de recursos informáticos en arquitecturas cliente-servidor (client-server).

1.1. Conceptos y características

El diseño de una arquitectura web robusta exige equilibrar requerimientos técnicos y funcionales para asegurar la continuidad operativa y el rendimiento (performance) del sistema bajo diversas condiciones de demanda.

Para comprender la estructura y el funcionamiento de una arquitectura web, es necesario reconocer los principales elementos que intervienen en la comunicación, el procesamiento de datos y la organización del sistema. A continuación, se presentan los conceptos fundamentales:

- **Cliente (client).** Entidad lógica (generalmente un navegador web como Chrome, Firefox o Edge, o una aplicación móvil) que realiza solicitudes de recursos mediante protocolos de red.
- **Servidor (server).** Sistema de cómputo o proceso informático que escucha de manera continua las peticiones entrantes, procesa la lógica de negocio asociada y retorna una respuesta formateada.

- **Patrón Modelo-Vista-Controlador (MVC).** Patrón arquitectónico que desacopla el software en tres capas fundamentales:
    - **Modelo:** gestiona la lógica de datos y las reglas de negocio.
    - **Vista**: renderiza la interfaz de usuario para la presentación de contenidos.
    - **Controlador:** actúa como intermediario, capturando las acciones del usuario en la Vista para invocar las operaciones del Modelo.

Las siguientes son las características clave de la arquitectura web:

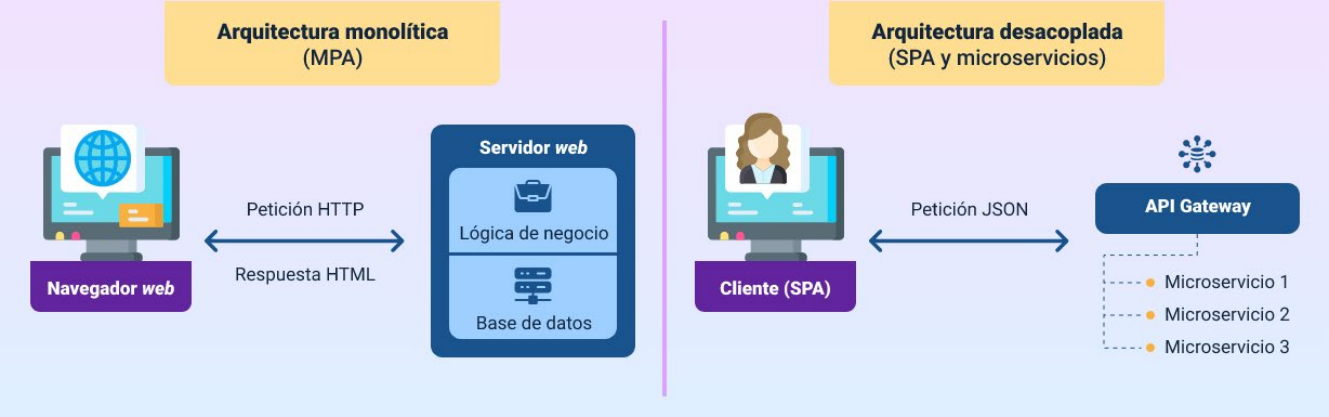
- **Escalabilidad.** Capacidad de una infraestructura para incrementar sus prestaciones ante una mayor demanda sin degradar la calidad del servicio. Se clasifica en:
    - **Escalabilidad vertical (scale up):** incremento de recursos físicos (procesador, memoria RAM, almacenamiento) en un único servidor.
    - **Escalabilidad horizontal (scale out):** adición de múltiples nodos o servidores interconectados bajo un balanceador de carga (load balancer).
- **Mantenibilidad y modularidad.** Capacidad de actualizar, depurar o refactorizar módulos de código de forma independiente, reduciendo el riesgo de efectos colaterales en la aplicación.
- **Disponibilidad y tolerancia a fallos**. Garantía de funcionamiento continuo del sistema mediante redundancia de componentes, copias de respaldo (backups) y estrategias de failover.
- **Desacoplamiento.** Grado de independencia entre los componentes del sistema, lo cual facilita la adopción de tecnologías diversas en cada capa.

El diseño de software moderno prioriza la alta cohesión dentro de cada módulo y el bajo acoplamiento entre módulos, lo que favorece la reutilización del código y la mantenibilidad.

1.2. Tipos de arquitectura web

La selección del tipo de arquitectura condiciona la estrategia de renderizado, la latencia de red y la complejidad en el despliegue del software. Para comprender estas diferencias, la siguiente figura presenta de manera comparativa las principales arquitecturas web, destacando su estructura y la forma en que se organizan sus componentes:

Figura 1. Esquema de arquitecturas web principales (monolítica vs. desacoplada con SPA)



- **Arquitectura monolítica (MPA)**
  - Navegador web.
  - Petición HTTP.
  - Respuesta HTML.

    - Servidor web.
    - Lógica de negocio.
    - Base de datos.
- **Arquitectura desacoplada (SPA y microservicios)**
    - Cliente (SPA).
    - Petición JSON.
    - API Gateway.
        - Microservicio 1.
        - Microservicio 2.
        - Microservicio 3.

A partir de la comparación presentada en la anterior figura, es posible profundizar en las características, el funcionamiento, las ventajas y las limitaciones de los principales tipos de arquitectura web. A continuación, se describen cada uno de estos enfoques:

- **Arquitectura monolítica (Multi-Page Application - MPA)**

En esta arquitectura, el código de la interfaz de usuario, la lógica de negocio y el acceso a la base de datos se compilan y despliegan dentro de una única unidad ejecutable. Cada interacción del usuario implica el procesamiento de una nueva petición HTTP en el servidor y la entrega de un documento HTML completo, generando una recarga visible de la página en el navegador.

    - **Ventajas:** sencillez en las etapas iniciales de desarrollo e indexación SEO nativa.

- **Desventajas:** alto consumo de ancho de banda por la transferencia repetida de activos y acoplamiento estricto.

- **Arquitectura desacoplada (Single Page Application - SPA)**

La aplicación SPA descarga, en la primera conexión, el código base en HTML, CSS y JavaScript. Las interacciones posteriores no recargan la página; en su lugar, se ejecutan peticiones asíncronas en segundo plano para obtener o enviar únicamente datos en formato estructurado (JSON o XML), dejando el trabajo de maquetación y renderizado al procesador del cliente.

- **Ventajas:** experiencia de usuario extremadamente fluida, similar a la de una aplicación nativa, y menor carga sobre el servidor.
- **Desventajas:** requiere configuraciones adicionales para el indexado en buscadores y presenta un mayor tiempo de carga inicial (initial load time).

- **Arquitectura basada en microservicios**

Es una evolución de la arquitectura orientada a servicios (SOA), en la cual la aplicación se descompone en un grupo de servicios pequeños, autónomos e independientemente desplegables. Cada microservicio ejecuta un proceso único, gestiona su propia base de datos y se comunica mediante protocolos ligeros (API REST o gRPC).

- **Ventaja**s: facilita la escalabilidad independiente de los servicios, el mantenimiento modular y el despliegue de actualizaciones sin afectar toda la aplicación**.**

- **Desventajas:** aumenta la complejidad de gestión, comunicación y monitoreo de los servicios, además de requerir una infraestructura más robusta para su implementación.

1.3. Mapas de navegación y procedimiento para diseñar

El mapa de navegación es la representación esquemática de la estructura de la información y los caminos de acceso que el usuario puede seguir dentro del sitio web. Según la forma en que se organizan y relacionan los contenidos, pueden establecerse diferentes estructuras de navegación, cada una adecuada para determinadas necesidades y recorridos del usuario. A continuación, se presentan los principales tipos:

- **Jerárquica (árbol).** Organizada en niveles de profundidad desde una página principal (home) hacia secciones y subsecciones. Es el modelo estándar para portales corporativos e institucionales.
- **Lineal.** Guía al usuario a través de una secuencia estricta de pasos. Utilizada en procesos de pago (checkout) o módulos de aprendizaje guiado.
- **En red (malla).** Permite la libre navegación entre nodos interconectados mediante hipervínculos, común en enciclopedias digitales y wikis.
- **Mixta.** Combina estructuras jerárquicas con enlaces transversales lineales o en red.

Una vez identificada la estructura de navegación más adecuada, es necesario organizar y representar los contenidos mediante un proceso que permita definir

jerarquías, relaciones y recorridos de usuario. Para ello, se propone el siguiente procedimiento:

- **Investigación y clasificación (card sorting).** Técnica participativa para agrupar y categorizar los contenidos de acuerdo con la mentalidad del usuario final.
- **Definición de la arquitectura de información (AI).** Creación del inventario completo de contenidos y definición de las jerarquías.
- **Diagramación del mapa de sitio (sitemap).** Elaboración del flujo visual que especifica la relación padre-hijo de cada URL o pantalla.
- **Bocetado de baja fidelidad (wireframing).** Diseño de esquemas de distribución espacial para verificar la ubicación de menús, migas de pan (breadcrumbs) y llamadas a la acción (CTA).
- **Validación de usabilidad.** Evaluación de los mapas de navegación para garantizar que el usuario pueda alcanzar cualquier contenido en un máximo de tres clics.

1.4. Técnicas de integración y sintaxis de contenidos web

La construcción del código fuente debe seguir los estándares internacionales del World Wide Web Consortium (W3C), asegurando que la estructura sea interpretable por cualquier agente de usuario. En este proceso, HTML5 permite organizar el contenido mediante etiquetas semánticas que identifican la función de cada bloque dentro del documento, evitando el uso indiscriminado de elementos genéricos como y favoreciendo una estructura más clara y organizada.

- **Estructuración semántica en HTML5**

Para apreciar la aplicación de estas etiquetas, el siguiente código presenta un ejemplo básico de estructuración semántica en HTML5:

```
<!DOCTYPE html>

<html lang="es">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Estructura Semántica Profesional</title>

</head>

<body>

<header>

<h1>Centro de Formación Técnica</h1>

<nav>

<ul>

<li><a href="#inicio">Inicio</a></li>

<li><a href="#cursos">Cursos</a></li>

</ul>

</nav>
```

```
</header>
<main>
<section id="cursos">
<article>
<h2>Desarrollo de Aplicaciones Web</h2>
<p>Contenido explicativo sobre la arquitectura del software...</p>
</article>
</section>
</main>
<footer>
<p>&copy; 2026 Servicio Nacional de Aprendizaje - SENA</p>
</footer>
</body>
</html>
```

Una vez definida la estructura semántica del contenido mediante HTML5, es necesario integrar los recursos encargados de la presentación visual y del comportamiento interactivo de la página. Para ello, CSS3 y JavaScript complementan la estructura del documento y permiten mantener una adecuada separación entre contenido, presentación y funcionalidad. Entre las principales técnicas de integración se encuentran:

- **Separación de capas.** Los estilos visuales se definen exclusivamente en archivos .css externos, mientras que la lógica de comportamiento se ubica en archivos .js.
- **Carga diferida (deferred loading).** Uso de los atributos async y defer en la importación de scripts para no bloquear el análisis del DOM durante la carga inicial.

1.5. Características y licenciamiento de la plataforma

El entorno donde se ejecutan y alojan los contenidos web exige analizar el tipo de licenciamiento técnico y legal para proyectar la viabilidad financiera del proyecto.

El uso de software bajo licencias de código abierto (open source) no excluye la existencia de costos operativos (OPEX), los cuales se desplazan desde el pago de derechos de uso hacia las tareas de infraestructura, auditoría de seguridad y soporte especializado. En este sentido, la selección de una plataforma requiere considerar aspectos como el costo de licencia, el acceso al código, la seguridad, las condiciones de despliegue y la flexibilidad de personalización. La siguiente tabla compara estos criterios en modelos de código abierto y plataformas propietarias o bajo modalidad SaaS:

Tabla 1. Cuadro comparativo de modelos de licenciamiento y plataformas

Criterio de evaluación	Licencia GPL y MIT	Licencia propietaria y SaaS
Costo de licencia.	Gratuita para modificación y distribución.	Pago por suscripción, volumen o transacción.

Criterio de evaluación	Licencia GPL y MIT	Licencia propietaria y SaaS
Acceso al código.	Total (source code visible y modificable).	Restringido (black box gestionado por el proveedor).
Seguridad.	Auditoría comunitaria; parches descentralizados.	Auditoría cerrada; acuerdos SLA garantizados.
Despliegue.	Requiere configuración de servidor propio o cloud.	Alojamiento administrado e integrado automáticamente.
Flexibilidad.	Personalización según requerimientos.	Sujeta a las extensiones permitidas por la plataforma.

2. Comunicación y diseño web

La comunicación en el entorno web constituye el conjunto de mecanismos, reglas y especificaciones técnicas que regulan la transferencia de información entre clientes y servidores a través de infraestructuras de red. Este flujo de datos debe complementarse con principios rigurosos de diseño visual, ergonomía digital y usabilidad, garantizando que la información se transmita de manera íntegra, eficiente y accesible para cualquier usuario e instrumento de navegación.

2.1. Técnicas de comunicación

El paradigma fundamental de la arquitectura web se basa en el modelo cliente-servidor (client-server). La interacción entre ambos extremos se realiza mediante peticiones y respuestas estructuradas. De acuerdo con el comportamiento de la interfaz y la sincronización en el intercambio de datos, las técnicas de comunicación se dividen en dos modalidades principales:

- **Comunicación síncrona**

La comunicación síncrona se caracteriza porque el cliente realiza una solicitud al servidor y espera a que este procese la petición y devuelva una respuesta antes de continuar con determinadas acciones. Durante este proceso existe una dependencia directa entre la solicitud realizada y la respuesta esperada, por lo que el flujo de interacción sigue una secuencia definida.

En aplicaciones web tradicionales, este modelo puede implicar el envío de una solicitud HTTP al servidor y la posterior carga de una nueva página o vista con la información procesada. Esto ocurre, por ejemplo, cuando un usuario completa un

formulario, lo envía y debe esperar la respuesta del servidor para visualizar el resultado. Aunque este mecanismo facilita procesos que requieren una secuencia controlada de acciones, las recargas completas de página pueden aumentar el tiempo de espera percibido y afectar la continuidad de la interacción.

- **Comunicación asíncrona**

La comunicación asíncrona permite que una aplicación web realice solicitudes al servidor sin que el usuario tenga que esperar necesariamente la finalización de cada petición para continuar interactuando con la interfaz. De esta manera, determinadas operaciones pueden ejecutarse en segundo plano mientras otras funciones de la aplicación permanecen disponibles.

Para implementar este tipo de comunicación pueden utilizarse tecnologías como XMLHttpRequest o la API Fetch, que permiten solicitar y recibir información del servidor sin recargar completamente la página. Los datos obtenidos, habitualmente en formatos como JSON, pueden ser procesados mediante JavaScript para actualizar únicamente los elementos necesarios de la interfaz. Esta modalidad favorece experiencias más dinámicas y fluidas, ya que permite actualizar información, consultar datos o ejecutar determinadas operaciones manteniendo la continuidad de la navegación.

2.2. Protocolos de comunicación

Un protocolo de comunicación es un conjunto formal de reglas, sintaxis y procedimientos que permiten organizar y controlar la transmisión de datos a través de una red. En la comunicación web, estos protocolos establecen la manera en que clientes y servidores intercambian solicitudes y respuestas. Entre los principales

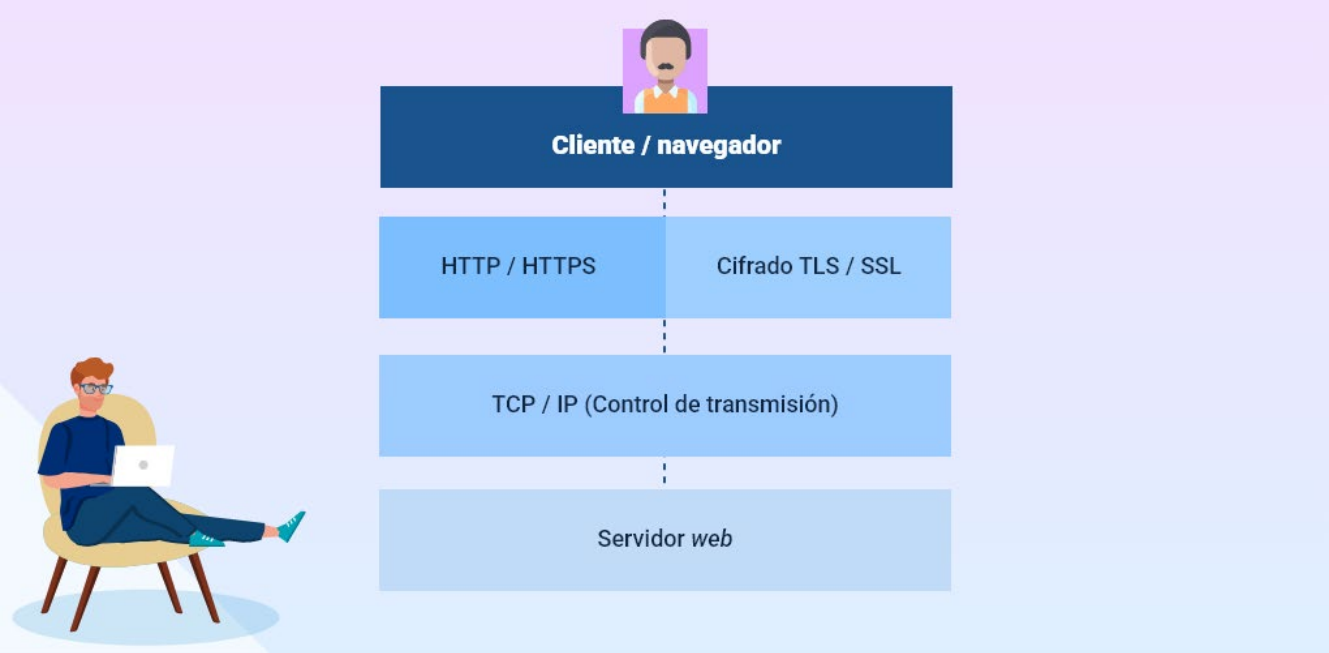
protocolos utilizados para este intercambio se encuentran HTTP y HTTPS, explicados a continuación:

a. Protocolo HTTP (Hypertext Transfer Protocol)

HTTP es un protocolo de la capa de aplicación que permite el intercambio de información entre clientes y servidores en la web. Su funcionamiento se basa en un modelo de petición-respuesta: el cliente, generalmente un navegador, envía una solicitud para acceder a un recurso o ejecutar una acción, y el servidor procesa la petición y devuelve una respuesta con la información correspondiente.

Una de sus principales características es que funciona como un protocolo sin estado (stateless), lo que significa que cada petición se procesa de manera independiente y, por defecto, el servidor no conserva información sobre las solicitudes anteriores. Para comprender cómo HTTP se integra con las diferentes capas que intervienen en la transmisión de datos entre el cliente y el servidor. La siguiente figura representa de manera esquemática este proceso de comunicación:

Figura 2. Diagrama de flujo y capas de los protocolos de comunicación



Las peticiones HTTP utilizan diferentes métodos para indicar la acción que el cliente desea realizar sobre un recurso del servidor. Cada método cumple una función específica dentro del intercambio de información y permite distinguir operaciones como consultar, enviar, actualizar o eliminar datos. Entre los métodos más utilizados se encuentran:

- **GET.** Solicita la representación de un recurso específico sin modificarlo en el servidor. Se utiliza principalmente para consultar y recuperar información.
- **POST.** Envía datos al servidor para la creación de un nuevo recurso o el procesamiento de entidades.

- **PUT.** Reemplaza de forma íntegra el recurso destino con la carga útil (payload) proporcionada.
- **PATCH.** Aplica modificaciones parciales a un recurso existente.
- **DELETE.** Elimina el recurso especificado en el servidor.

Una vez procesada la petición, el servidor devuelve una respuesta que incluye un código de estado HTTP. Este código permite identificar de manera estandarizada el resultado de la solicitud y se organiza en cinco categorías principales según el tipo de respuesta:

- **1xx (Informativos).** Indican que la petición ha sido recibida y el proceso continúa.
- **2xx (Éxito).** Confirman que la acción fue recibida, entendida y aceptada con éxito (ej. 200 OK, 201 Created).
- **3xx (Redirección).** Indican que el cliente debe tomar medidas adicionales para completar la solicitud (ej. 301 Moved Permanently, 304 Not Modified).
- **4xx (Errores del** cliente**).** Señalan fallos atribuibles a la petición del usuario (ej. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).
- **5xx (Errores del servidor).** Indican que el servidor falló al procesar una petición válida (ej. 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable).

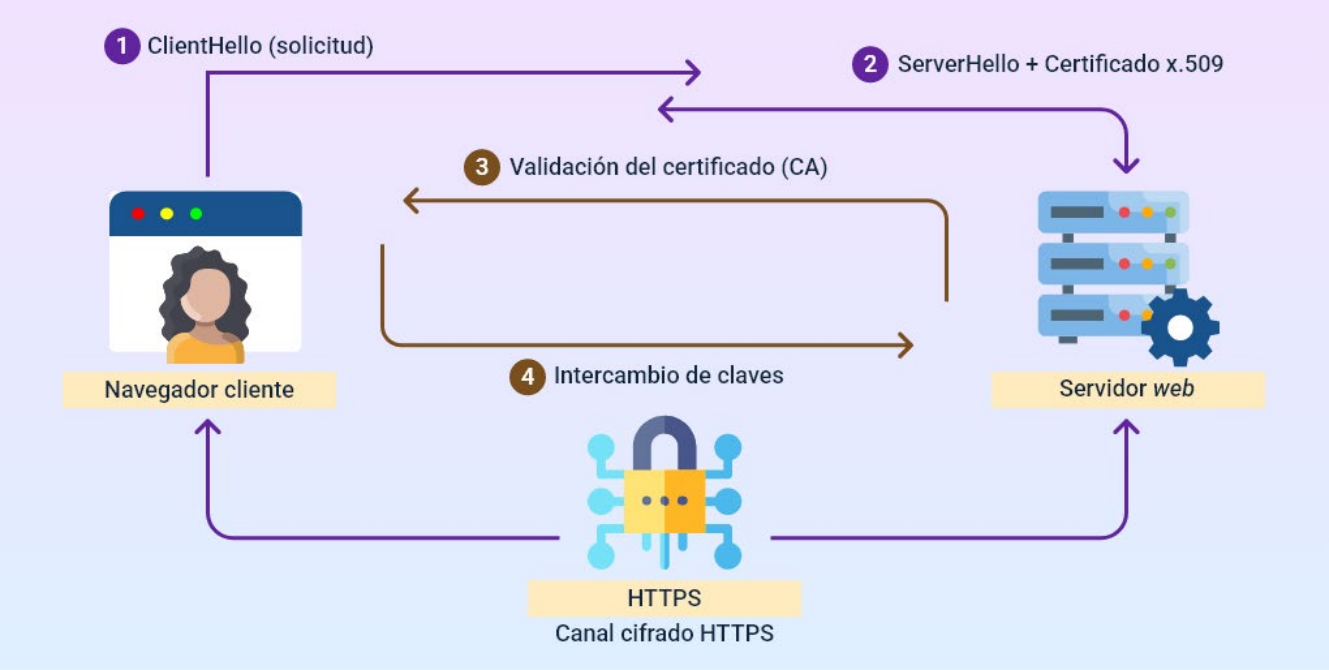
b. Protocolo HTTPS (Hypertext Transfer Protocol Secure)

HTTPS es la implementación segura del protocolo HTTP y utiliza el protocolo Transport Layer Security (TLS), sucesor de Secure Sockets Layer (SSL), para proteger la

información intercambiada entre el cliente y el servidor. Este mecanismo permite cifrar los datos transmitidos, verificar la identidad del servidor mediante certificados digitales y preservar la integridad de la información durante la comunicación.

Antes de establecer el canal cifrado, el cliente y el servidor realizan un proceso de negociación en el que intercambian información, validan el certificado digital y establecen los parámetros necesarios para proteger la conexión. La siguiente figura representa las principales etapas de este proceso:

Figura 3. Diagrama de secuencia del apretón de manos HTTPS



- **Pilares de seguridad en HTTPS**

La seguridad proporcionada por HTTPS se sustenta en tres pilares fundamentales: el cifrado, la integridad y la autenticación. El cifrado de datos protege la información intercambiada entre el cliente y el servidor, de manera que credenciales, datos

personales u otra información sensible no puedan ser interpretados fácilmente si son interceptados durante la transmisión.

La integridad de los datos permite verificar que la información no haya sido modificada durante su recorrido entre el cliente y el servidor. Los mecanismos criptográficos utilizados por TLS permiten detectar alteraciones en los datos transmitidos y evitar que información manipulada sea aceptada como válida.

Por su parte, la autenticación permite al cliente verificar la identidad del servidor mediante certificados digitales emitidos por autoridades de certificación (CA) de confianza. Este proceso contribuye a confirmar que la comunicación se establece con el servidor esperado y no con un tercero que intenta suplantarlo.

De esta manera, el cifrado, la integridad y la autenticación actúan de forma complementaria para proteger la comunicación HTTPS y proporcionar un intercambio de información más seguro en las aplicaciones web.

2.3. Características del diseño web

El diseño web moderno se fundamenta en la confluencia entre la usabilidad, la experiencia de usuario (UX), la interfaz de usuario (UI) y el cumplimiento normativo de accesibilidad universal. En este contexto, el diseño web debe responder tanto a la diversidad de dispositivos y tamaños de pantalla como a las necesidades de acceso de los usuarios. Para ello, se consideran dos características fundamentales:

a. Diseño web responsivo (responsive web design)

Consiste en la capacidad de una maquetación para reestructurar sus componentes de manera fluida en función del espacio disponible en pantalla, utilizando hojas de estilo CSS3, rejillas flexibles (css grid), cajas flexibles (flexbox) y reglas de consulta de medios (media queries).

- **Enfoque mobile-first**

Este enfoque es una estrategia de diseño y desarrollo que prioriza inicialmente la maquetación para dispositivos móviles de pantallas reducidas y agrega complejidad visual y estructural de forma progresiva a medida que aumenta el tamaño de la pantalla (progressive enhancement). En CSS, este enfoque puede implementarse mediante estilos base para dispositivos móviles y consultas de medios (media queries) que modifican la disposición de los elementos en pantallas de mayor tamaño, como se presenta en el siguiente ejemplo:

```
/* Estilos Base: Mobile-First */

.contenedor-principal {

display: flex;

flex-direction: column;

width: 100%;

padding: 1rem;

}

/* Punto de interrupción (Breakpoint) para tabletas */
```

```
@media (min-width: 768px) {
.contenedor-principal {
flex-direction: row;
gap: 2rem;
}
}
/* Punto de interrupción para pantallas de escritorio */
@media (min-width: 1200px) {
.contenedor-principal {
max-width: 1200px;
margin: 0 auto;
}
}
```

b. Accesibilidad web (Web Content Accessibility Guidelines - WCAG)

Las pautas de accesibilidad para el contenido web, desarrolladas por el W3C, establecen criterios orientados a garantizar que los sitios web sean utilizables por personas con diversas discapacidades, incluidas las visuales, auditivas, motoras y cognitivas. Estos criterios se organizan en cuatro principios fundamentales, conocidos por el acrónimo POUR:

- **Perceptible.** La información y los componentes de la interfaz deben ser presentados de modo que los usuarios puedan percibirlos (por ejemplo, proporcionando alternativas de texto mediante atributos alt en imágenes).
- **Operable.** La interfaz no puede requerir interacciones que el usuario no pueda realizar; toda la funcionalidad debe ser accesible mediante el teclado sin atrapamiento del foco (keyboard trap).
- **Comprensible.** La información y el funcionamiento de la interfaz de usuario deben ser comprensibles y predecibles.
- **Robusto.** El contenido debe ser lo suficientemente robusto para ser interpretado de forma fiable por una amplia variedad de agentes de usuario, incluyendo tecnologías de asistencia (screen readers).

Para cumplir con los criterios de contraste correspondientes al nivel AA de las WCAG, la relación de contraste entre el texto y su fondo debe ser de al menos 4,5:1 para texto normal y de 3:1 para texto de gran tamaño.

En conjunto, los principios de comunicación, seguridad, adaptabilidad y accesibilidad permiten establecer criterios técnicos para evaluar el funcionamiento y la experiencia de uso de un sitio web. Estos criterios pueden verificarse mediante parámetros relacionados con la seguridad de la red, el tiempo de respuesta, el contraste visual, la adaptabilidad de la interfaz y la navegabilidad. La siguiente tabla sintetiza estos aspectos y presenta algunos parámetros de referencia para su evaluación:

Tabla 2. Directrices técnicas de comunicación y diseño web

Dimensión técnica	Parámetro de evaluación	Estándar / recomendación técnica	Frecuencia de verificación
Seguridad de red.	Certificados de cifrado.	TLS 1.3 y renovación automática mediante Let's Encrypt.	Continuo.
Tiempo de respuesta.	Time to First Byte (TTFB).	Menor a 800 milisegundos en peticiones GET.	Por cada despliegue.
Contraste de color.	Relación de luminancia.	Mínimo 4,5:1 (cumplimiento WCAG AA).	Auditoría de interfaz.
Adaptabilidad UI.	Puntos de interrupción.	Configuración de viewports para 320 px, 768 px, 1024 px y 1440 px.	Fase de maquetación.
Navegabilidad.	Jerarquía del teclado.	Atributos tabindex lógicos y marcos de foco (focus ring) visibles.	Pruebas de usabilidad.

3. Desarrollo web

El desarrollo web abarca la fase de implementación práctica del ciclo de vida del software, integrando la lógica de programación del lado del cliente (frontend), la lógica del servidor (backend), el manejo de bases de datos y la orquestación de herramientas especializadas para la edición, el empaquetado y la integración continua.

3.1. Técnicas de desarrollo web

La construcción de software web se fundamenta en la separación de responsabilidades (Separation of Concerns, SoC), principio que permite organizar los diferentes componentes de una aplicación de acuerdo con las funciones que cumplen. Esta separación facilita distinguir los elementos relacionados con la estructura, la presentación y la interacción del usuario de aquellos encargados de la lógica de negocio, la seguridad y la gestión de los datos. Desde esta perspectiva, el desarrollo web se aborda mediante dos áreas principales:

a. Desarrollo frontend

El desarrollo frontend comprende los componentes de una aplicación web que se ejecutan en el navegador y con los que el usuario interactúa directamente. Su construcción integra la estructura del contenido, la presentación visual y el comportamiento interactivo de la interfaz. Para comprender cómo se desarrollan estos componentes, se presentan las principales técnicas y tecnologías empleadas en el frontend:

- **Estructuración semántica (HTML5).** Definición de la jerarquía conceptual del documento mediante elementos y etiquetas semánticas que organizan

el contenido de forma lógica, accesible y estructurada, facilitando su interpretación por navegadores, motores de búsqueda y tecnologías de asistencia.

- **Estilizado y maquetación avanzada (CSS3).** Definición del comportamiento sintáctico y estético mediante metodologías como BEM (Block Element Modifier) o arquitecturas CSS atómicas.
- **Lógica de cliente interactiva (JavaScript ES6+).** Programación imperativa o funcional para gestionar el estado de la interfaz, el procesamiento de eventos, la validación de formularios en el cliente y la manipulación del DOM.

b. Desarrollo backend

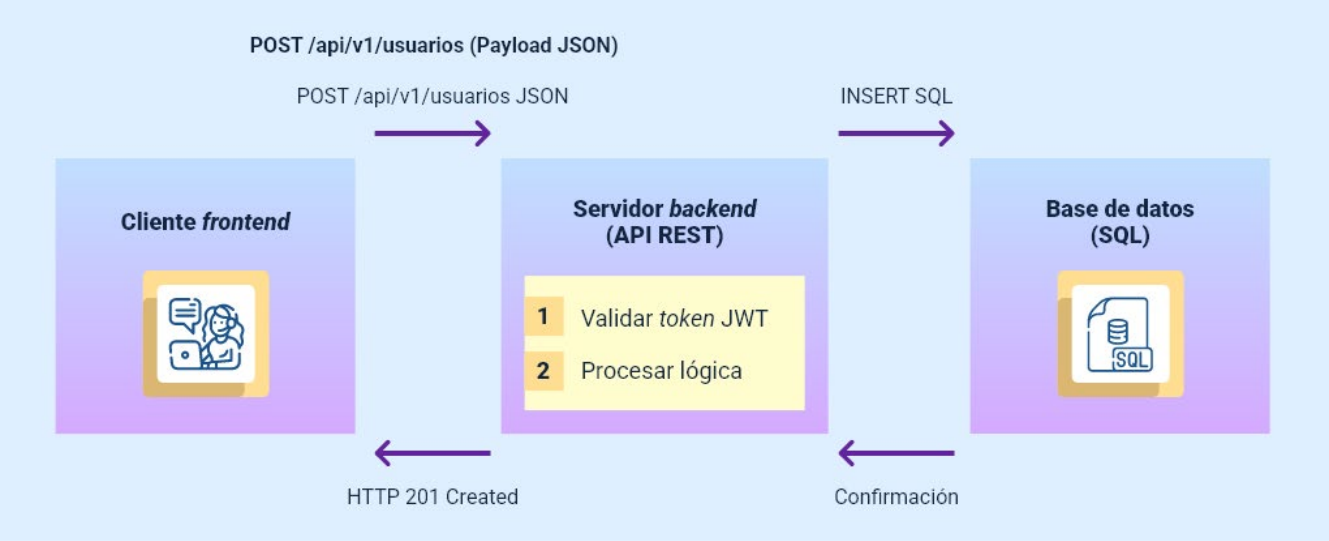
El desarrollo backend comprende los procesos que se ejecutan en el servidor y que permiten gestionar la lógica de negocio, la seguridad de acceso, la autenticación de usuarios, el procesamiento de solicitudes y la persistencia de datos. Su implementación se apoya en diferentes tecnologías y enfoques que permiten estructurar y gestionar estas funciones. Entre los principales se encuentran:

- **Lenguajes de programación del servidor.** Utilización de lenguajes y entornos como Node.js, Python (Django / FastAPI), PHP, Java (Spring Boot) o C# (.NET Core) para implementar la lógica de negocio, procesar solicitudes, gestionar datos y ejecutar las operaciones requeridas por las aplicaciones web.
- **Estilo arquitectónico REST (Representational State Transfer).** Diseño de servicios web que exponen recursos mediante URLs unívocas y

operaciones estándar asociadas a los verbos HTTP, utilizando JSON como formato preferente para el intercambio de información.

La interacción entre estos componentes puede apreciarse en el flujo de una solicitud desde el cliente hacia el servidor, donde la API REST procesa la petición, ejecuta la lógica correspondiente y se comunica con la base de datos para gestionar la información. La siguiente figura representa de manera esquemática este proceso y el intercambio de respuestas entre los diferentes componentes:

Figura 4. Diagrama de interacción cliente - API REST - base de datos



- **POST /api/v1/usuarios (Payload JSON)**
  - Cliente frontend
  - POST /api/v1/usuarios JSON)
- **Servidor backend (API REST)**
  - Validar token JWT
  - Procesar lógica
  - INSERT SQL

- Confirmación
- Base de datos (SQL)

- **HTTP 201 Created**

3.2. Herramientas de desarrollo web

El flujo de trabajo moderno exige un ecosistema de herramientas especializadas para optimizar la productividad del desarrollador y la calidad del código entregado.

La siguiente es la cadena y flujo de trabajo con herramientas de desarrollo web:

a. Editores de código (VS Code)

Herramientas de edición de código con soporte para resaltado de sintaxis, autocompletado inteligente, refactorización y depuración integrada (p. ej., Visual Studio Code y JetBrains WebStorm).

b. Control de versiones (Git y GitHub)

Herramientas y plataformas que permiten registrar y gestionar los cambios realizados en el código fuente a lo largo del tiempo, facilitando la colaboración, la trazabilidad y el trabajo mediante ramas (branches).

c. Empaquetador (Vite o Webpack)

Herramientas que permiten gestionar y optimizar los recursos de una aplicación durante el proceso de desarrollo y construcción, incluyendo tareas como el procesamiento de dependencias, la transformación del código y la generación de archivos optimizados para su distribución.

d. Contenedor (Docker)

Tecnología de contenerización que permite empaquetar una aplicación junto con sus dependencias y configuraciones en un entorno aislado y reproducible, facilitando su ejecución consistente en diferentes entornos de desarrollo, prueba y producción.

e. Servidor de producción

Entorno en el que la aplicación se despliega y queda disponible para los usuarios finales. Comprende la configuración del servidor, la ejecución de los servicios necesarios y la gestión de aspectos como disponibilidad, seguridad, rendimiento y monitoreo.

Es importante destacar que, dentro de este flujo de trabajo, el control de versiones permite gestionar de manera organizada los cambios realizados durante el desarrollo de una aplicación. Como ejemplo de su aplicación, los siguientes comandos muestran un flujo básico de trabajo con Git, desde la inicialización de un repositorio local y la creación de una rama hasta el registro de los cambios y su envío al repositorio remoto:

```
# Inicializar un nuevo repositorio local

git init

# Crear una nueva rama para una característica específica

git checkout -b feature/autenticacion-jwt

# Registrar cambios en la zona de preparación (Staging Area)

git add .
```

```
# Confirmar los cambios en el historial local con un mensaje descriptivo
git commit -m "feat: implementar validación de tokens JWT en el módulo de login"
# Vincular y enviar la rama al repositorio remoto
git push origin feature/autenticacion-jwt
```

3.3. Características de las herramientas

Las herramientas de desarrollo web se diferencian por la función que desempeñan dentro del proceso de construcción de una aplicación y por las características técnicas que aportan al flujo de trabajo. Mientras algunas se orientan al control y trazabilidad de los cambios en el código, otras facilitan la construcción y optimización de los recursos, la estandarización de los entornos, el análisis de la calidad del código o la validación de servicios y APIs.

Estas herramientas también favorecen la automatización y la consistencia de los procesos de desarrollo. Por ejemplo, los sistemas de control de versiones permiten conservar el historial de cambios y apoyar el trabajo colaborativo; las herramientas de construcción procesan y optimizan los recursos de una aplicación; y los contenedores permiten reproducir configuraciones y dependencias en diferentes entornos. A estas se suman herramientas destinadas al análisis estático del código y a las pruebas de APIs, que contribuyen a detectar errores y verificar el comportamiento de los servicios antes de su despliegue.

Por esta razón, la selección de herramientas no depende únicamente de la tecnología utilizada para desarrollar la aplicación, sino también de aspectos como la trazabilidad, la portabilidad, la automatización, la calidad del código y las necesidades de prueba. La siguiente tabla presenta una comparación de algunas herramientas representativas del ecosistema de desarrollo web, organizadas según su categoría, función principal y característica clave:

Tabla 3. Comparativa de herramientas del ecosistema de desarrollo

Herramienta	Categoría	Función principal	Característica clave
Git.	Control de versiones.	Seguimiento de cambios en código.	Arquitectura distribuida e historial inmutable mediante *hashes* SHA-1.
Docker.	Contenedorización.	Empaquetado de entorno y dependencias.	Portabilidad total mediante el aislamiento de procesos del sistema operativo.
Vite.	Build tool / bundler.	Servidor de desarrollo y empaquetado.	Reemplazo de módulos en caliente (HMR) ultra rápido usando ES Modules nativos.
ESLint.	Analizador estático (linter).	Detección de errores y patrones anómalos.	Forzado de guías de estilo de código automatizadas en el equipo de trabajo.

Herramienta	Categoría	Función principal	Característica clave
Postman.	Pruebas de API.	Inspección y prueba de endpoints REST y gRPC.	Generación automatizada de documentación de APIs y ejecución de colecciones de prueba.

4. Sitio web

Un sitio web es una colección estructurada de documentos de hipertexto, hojas de estilo, scripts y activos multimedia interconectados de forma lógica, alojados en una infraestructura de servidores y accesibles a través de Internet mediante los protocolos HTTP o HTTPS.

4.1. Definición y características

Para que una solución web alcance condiciones adecuadas de calidad y funcionamiento, debe integrar características relacionadas con su identificación, disponibilidad y capacidad de operar en diferentes entornos tecnológicos. Estos aspectos favorecen el acceso a los recursos, la continuidad del servicio y una experiencia consistente para los usuarios, independientemente del navegador o sistema operativo utilizado. Entre las principales características se encuentran:

- **Identificación e indexabilidad.** Direccionamiento inequívoco mediante nombres de dominio y resolución DNS, junto con una estructura del sitio que facilite su exploración e indexación por los motores de búsqueda.
- **Disponibilidad operativa.** Capacidad del sitio para mantenerse accesible y en funcionamiento durante un periodo determinado, generalmente expresada mediante un porcentaje de disponibilidad (uptime). Los niveles esperados pueden establecerse de acuerdo con los requisitos del servicio y los acuerdos de nivel de servicio (SLA).
- **Interoperabilidad multiplataforma.** Capacidad de un sitio web para mantener un funcionamiento y una experiencia de uso consistentes en

diferentes navegadores (cross-browser compatibility), dispositivos y sistemas operativos.

4.2. Tipos y categorías

Los sitios web pueden clasificarse de acuerdo con su propósito, las funcionalidades que ofrecen y el tipo de interacción que establecen con sus usuarios. Esta clasificación permite diferenciar sitios orientados principalmente a la difusión de información de aquellos que incorporan procesos transaccionales, educativos o aplicaciones que proporcionan servicios directamente desde el navegador. Aunque un mismo sitio puede combinar características de varias categorías, es posible identificar algunos tipos principales según su función predominante. Entre ellos se encuentran:

- **Portales institucionales o corporativos.** Orientados a la proyección de la identidad corporativa, la divulgación de información estratégica y la comunicación con sus públicos, facilitando el acceso a servicios, información institucional y canales de atención.
- **Sistemas de comercio electrónico (E-commerce).** Diseñados para la gestión de catálogos de productos, carritos de compras, integración de pasarelas de pago cifradas y control de inventarios en tiempo real.
- **Sistemas de gestión de aprendizaje (LMS).** Plataformas especializadas en el despliegue de contenidos formativos, administración de aulas virtuales, evaluación y seguimiento del progreso académico.
- **Aplicaciones** web **y servicios SaaS (Software as a Service).** Soluciones que proporcionan funcionalidades y servicios de software accesibles

mediante la web, como editores de documentos, sistemas de gestión empresarial y herramientas colaborativas.

4.3. Características de los elementos

Un sitio web se estructura mediante diferentes elementos que organizan la información y facilitan la navegación e interacción del usuario. Cada uno cumple una función específica dentro de la interfaz y contribuye a establecer una jerarquía visual y funcional del contenido. Aunque su distribución puede variar según el propósito y diseño del sitio, generalmente se identifican elementos como los siguientes:

- **Encabezado (header / nav).** Se ubica generalmente en la parte superior y reúne elementos de identificación y navegación, como el logotipo y el menú principal.
- **Sección principal (hero banner).** Área destacada que presenta información relevante, mensajes principales o llamadas a la acción.
- **Área de contenido (main / articles).** Espacio central destinado a organizar y presentar la información principal del sitio mediante textos, imágenes, artículos u otros recursos.
- **Barra lateral (sidebar).** Sección complementaria que puede contener enlaces, menús secundarios, información adicional u otros elementos de apoyo.
- **Pie de página (footer).** Área ubicada al final del sitio que suele agrupar información complementaria, enlaces de interés, datos de contacto o información legal.

4.4. Requerimientos funcionales y no funcionales

La ingeniería de requerimientos permite identificar, analizar y documentar las necesidades que debe satisfacer un sitio o aplicación web. Durante este proceso se establecen las funciones que deberá ejecutar el sistema, las condiciones necesarias para su operación y los criterios que permitirán comprobar posteriormente si la solución desarrollada responde a las necesidades definidas. Los requerimientos pueden agruparse principalmente en funcionales y no funcionales:

- **Requerimientos funcionales (RF)**

Definen los comportamientos, servicios e interacciones que el sistema debe proporcionar en respuesta a las acciones del usuario o a determinados eventos. Pueden incluir funciones como registrar usuarios, autenticar credenciales, realizar búsquedas, procesar formularios, gestionar información o ejecutar transacciones. Por ejemplo, un requerimiento funcional puede establecer: «El sistema debe permitir al usuario recuperar su contraseña mediante el correo electrónico registrado».

- **Requerimientos no funcionales (RNF)**

Establecen las condiciones y atributos de calidad bajo los cuales deben ejecutarse las funcionalidades del sistema. Incluyen aspectos relacionados con rendimiento, seguridad, disponibilidad, accesibilidad, compatibilidad, escalabilidad y usabilidad. Por ejemplo, un requerimiento no funcional puede definir un tiempo máximo de respuesta, un porcentaje de disponibilidad o un determinado criterio de accesibilidad.

Tanto los requerimientos funcionales como los no funcionales deben formularse de manera clara, verificable y, cuando corresponda, medible, de modo que puedan establecerse criterios objetivos de aceptación. Por esta razón, expresiones ambiguas

como «el sitio debe ser rápido» deben sustituirse por criterios cuantificables asociados al rendimiento, indicando la métrica, el valor esperado y las condiciones en las que será evaluada.

4.5. Parámetros de configuración, descripción y palabras clave

Los parámetros de configuración de un sitio web permiten establecer propiedades que influyen en su correcta visualización, identificación, compatibilidad y presentación de información ante navegadores y otros servicios digitales. Estas configuraciones pueden definirse tanto en el documento HTML como en otros componentes de la aplicación, dependiendo de las características técnicas y los requerimientos establecidos para el proyecto.

En HTML, una parte de esta configuración se incorpora en el elemento <head>, donde se incluyen metadatos que proporcionan información sobre el documento sin formar parte directamente del contenido visible de la página. Estos datos permiten especificar aspectos como la codificación de caracteres, la adaptación a diferentes tamaños de pantalla, la descripción del contenido, la autoría y la información utilizada al compartir el sitio en determinadas plataformas.

El siguiente fragmento de código ejemplifica la configuración de algunos de estos parámetros mediante etiquetas <meta>, propiedades Open Graph y el elemento <title>:

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="Componente formativo sobre fundamentos de desarrollo web, arquitecturas cliente-servidor y protocolos de comunicación para aprendices SENA.">
<meta name="keywords" content="desarrollo web, HTML5, CSS3, JavaScript, interoperabilidad, protocolos, SENA">
<meta name="author" content="Servicio Nacional de Aprendizaje - SENA">
<!-- Protocolo Open Graph para redes sociales -->
<meta property="og:title" content="Fundamentos para el desarrollo de aplicaciones web">
<meta property="og:description" content="Aprenda las técnicas de interoperabilidad y protocolos técnicos en desarrollo web.">
<meta property="og:type" content="website">
<title>Fundamentos para el Desarrollo de Aplicaciones Web | SENA</title>
</head>
```

En este ejemplo, los parámetros incluidos en el <head> permiten definir información básica para la interpretación y presentación del documento web. Sin embargo, la configuración técnica de un sitio también debe responder a los requerimientos establecidos durante su desarrollo, de manera que aspectos

funcionales, de seguridad, rendimiento y capacidad puedan verificarse mediante criterios de aceptación definidos. En relación con los requerimientos abordados previamente, la siguiente tabla presenta algunos ejemplos y sus respectivas métricas de aceptación:

Tabla 4. Clasificación de requerimientos de un sitio web

Categoría	Tipo de requerimiento	Descripción técnica	Métrica de aceptación
Funcional.	Autenticación.	Inicio de sesión con correo institucional.	Acceso autorizado con credenciales válidas y rechazo con credenciales inválidas.
Funcional.	Exportación de datos.	Generación de reportes en formato PDF y Excel.	Descarga correcta del archivo generado.
No funcional.	Seguridad.	Cifrado obligatorio de contraseñas.	Algoritmo Argon2id o bcrypt con salt.
No funcional.	Rendimiento.	Puntuación en prueba de velocidad.	Puntuación > 90 / 100 en Google Lighthouse.
No funcional.	Escalabilidad.	Carga masiva de usuarios concurrentes.	Soporte para 5000 peticiones por minuto.

5. Análisis de información

El análisis de información en el ámbito del desarrollo web es un proceso sistemático que comprende la recolección, el procesamiento, la interpretación y la evaluación de datos cuantitativos y cualitativos derivados del funcionamiento de una aplicación y de la interacción de los usuarios con ella. Este proceso permite identificar patrones de comportamiento, detectar problemas técnicos y obtener evidencias que orienten la mejora continua del sitio o aplicación web.

5.1. Concepto de análisis de información

El análisis de información consiste en transformar los datos obtenidos del funcionamiento de un sitio o aplicación web en información organizada que pueda ser interpretada y utilizada para la toma de decisiones. Estos datos pueden proceder de diferentes fuentes, como registros de transacciones, métricas de navegación, mapas de calor, tiempos de carga, eventos de interacción y registros de errores HTTP.

En el desarrollo web, su propósito es comprender tanto el comportamiento técnico de la aplicación como la forma en que los usuarios interactúan con sus contenidos y funcionalidades. Por ejemplo, el análisis de los tiempos de carga puede evidenciar problemas de rendimiento, mientras que las métricas de navegación pueden revelar rutas frecuentes, puntos de abandono o dificultades durante la interacción.

La información obtenida permite sustentar decisiones relacionadas con la optimización del código, la mejora de la usabilidad, la corrección de errores y el seguimiento de objetivos técnicos o de negocio. De esta manera, el análisis proporciona evidencia para orientar acciones de mejora sobre el sitio o aplicación web.

El análisis de información convierte los datos disponibles en evidencia útil para comprender el comportamiento de una aplicación y fundamentar decisiones de mejora.

5.2. Procedimientos de análisis

El análisis de información requiere un procedimiento organizado que permita trabajar los datos de manera progresiva, desde su obtención hasta su utilización en acciones de mejora. Cada etapa cumple una función específica y debe responder a los objetivos técnicos o funcionales que se desean evaluar.

Las fuentes de información pueden incluir registros del servidor, herramientas de analítica, métricas de rendimiento, eventos de interacción y registros de errores. La selección de los datos depende del propósito del análisis; por ejemplo, un estudio de rendimiento puede centrarse en tiempos de respuesta y carga, mientras que un análisis de interacción puede considerar recorridos y acciones realizadas por los usuarios.

Este procedimiento se desarrolla como un ciclo continuo, ya que los resultados obtenidos permiten implementar modificaciones y posteriormente evaluar sus efectos mediante nuevos datos. Para organizar este proceso, se consideran cuatro fases fundamentales:

a. Recolección de datos

Esta fase consiste en obtener la información necesaria para evaluar el comportamiento de la aplicación web. Los datos pueden proceder de registros del servidor, herramientas de analítica, sistemas de monitoreo, métricas de rendimiento y registros de errores. Su selección debe responder al objetivo del análisis para asegurar que la información recopilada sea pertinente y útil.

b. Depuración y filtrado

Los datos recopilados deben revisarse y organizarse antes de ser analizados. Esta fase permite identificar registros incompletos, duplicados, inconsistentes o irrelevantes y seleccionar únicamente la información necesaria. También pueden aplicarse filtros relacionados con periodos de tiempo, dispositivos, páginas, eventos o tipos de solicitudes.

c. Interpretación de KPI

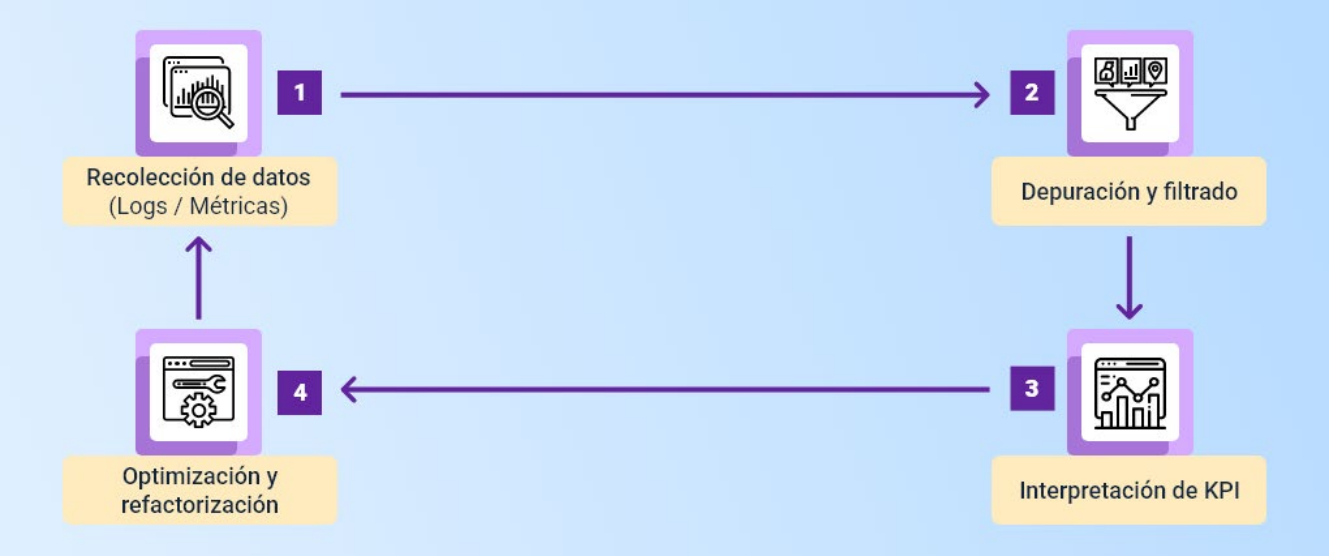
La información procesada se analiza mediante indicadores clave de rendimiento (Key Performance Indicators, KPI) relacionados con los objetivos establecidos. Estos indicadores permiten reconocer tendencias, comparar resultados e identificar posibles desviaciones. Su interpretación debe considerar el contexto y la relación entre diferentes métricas para determinar qué aspectos requieren atención o mejora.

d. Optimización y refactorización

A partir de los resultados del análisis se establecen acciones para corregir problemas o mejorar el funcionamiento de la aplicación. Estas pueden incluir la optimización de recursos, la corrección de errores, la mejora de procesos o la refactorización del código. Una vez implementados los cambios, sus efectos pueden evaluarse mediante una nueva recolección y análisis de datos.

Las fases descritas se articulan de manera secuencial y cíclica, de modo que los datos avanzan desde su recolección y depuración hasta su interpretación y posterior utilización en acciones de optimización. Una vez implementadas las mejoras, el proceso puede reiniciarse con la recopilación de nuevos datos para evaluar los resultados obtenidos. La siguiente figura representa gráficamente esta relación:

Figura 5. Diagrama del ciclo metodológico de análisis web



5.3. Modelos de resultados

Los resultados obtenidos durante el análisis de información deben organizarse de manera que faciliten su interpretación y permitan relacionar los datos técnicos con el comportamiento de los usuarios. Para ello, pueden utilizarse tableros de control (dashboards), informes ejecutivos y reportes de seguimiento que integren indicadores relevantes para los objetivos del sitio o aplicación web.

La presentación estructurada de los resultados permite comparar periodos, detectar desviaciones, analizar tendencias y valorar el efecto de las acciones implementadas. Dependiendo del propósito del análisis, los indicadores pueden agruparse en dos grandes categorías:

a. **Métricas de rendimiento técnico**

Estas métricas permiten evaluar aspectos relacionados con la velocidad de carga, la capacidad de respuesta y la estabilidad visual de una página web. Entre las principales métricas se encuentran:

- **Largest Contentful Paint (LCP).** Mide el tiempo necesario para renderizar el elemento visual de mayor tamaño en la pantalla. Debe ser inferior a 2,5 segundos.
- **Interaction to Next Paint (INP).** Mide la capacidad de respuesta de una página ante las interacciones del usuario, como clics, toques o pulsaciones de teclado. Se considera adecuado un valor de hasta 200 milisegundos.
- **Cumulative Layout Shift (CLS).** Mide la estabilidad visual evitando desplazamientos inesperados de elementos en pantalla. Su valor debe ser menor a 0,1.

b. **Métricas de comportamiento e interacción**

Estas métricas permiten analizar la forma en que los usuarios navegan por el sitio, utilizan sus funcionalidades y responden a los contenidos o acciones propuestas. Entre las métricas más utilizadas se encuentran:

- **Tasa de rebote (bounce rate).** Porcentaje de sesiones en las que el usuario abandona el sitio desde la página de entrada sin realizar ninguna interacción secundaria.
- **Tasa de conversión (conversion rate).** Porcentaje de usuarios que completan con éxito una acción objetivo predefinida (por ejemplo, registrarse en un curso o completar una compra).

5.4. Técnicas de evaluación

Las técnicas de evaluación permiten comprobar si los cambios implementados producen los resultados esperados. A diferencia de las métricas, que describen el comportamiento de la aplicación o de los usuarios, estas técnicas permiten realizar comparaciones o someter el sistema a determinadas condiciones para evaluar su respuesta.

Su aplicación puede orientarse tanto a la experiencia del usuario como al desempeño técnico de la infraestructura. En este contexto, se consideran dos técnicas principales:

- **Pruebas A/B (A/B testing)**

Son experimentos controlados que comparan dos versiones de una misma página, interfaz o elemento web. Una versión actúa como referencia y la otra incorpora una modificación que se desea evaluar. Ambas se muestran a grupos comparables de usuarios y sus resultados se analizan mediante indicadores previamente definidos, como conversiones, clics o finalización de una acción.

- **Pruebas de carga y estrés (load and stress testing)**

Permiten evaluar el comportamiento técnico de una aplicación ante diferentes niveles de demanda. Las pruebas de carga analizan el funcionamiento del sistema bajo volúmenes previstos o progresivos de solicitudes, mientras que las pruebas de estrés incrementan la demanda hasta superar las condiciones normales para identificar límites de capacidad, cuellos de botella o posibles fallos.

Estas pruebas pueden ejecutarse mediante herramientas especializadas, como Apache JMeter o k6, que permiten simular solicitudes concurrentes y observar indicadores como tiempos de respuesta, errores y uso de recursos.

5.5. Conclusiones

El análisis de información constituye un proceso fundamental para la mejora continua de los sitios y aplicaciones web, ya que permite fundamentar las decisiones técnicas y funcionales en datos obtenidos durante su operación. El análisis sistemático de métricas facilita reconocer problemas de rendimiento, estabilidad, accesibilidad e interacción, así como establecer prioridades de intervención de acuerdo con los resultados identificados.

Este enfoque permite que las acciones de mejora no respondan únicamente a apreciaciones subjetivas, sino a evidencias que puedan analizarse y compararse. La optimización de recursos, la refactorización del código, los ajustes de infraestructura o las modificaciones de la interfaz deben relacionarse con indicadores y criterios previamente definidos que permitan comprobar posteriormente el efecto de las decisiones adoptadas.

De esta manera, el análisis, la evaluación y la optimización conforman un ciclo de mejora continua en el que los resultados obtenidos orientan nuevas acciones y permiten realizar posteriores mediciones. La selección de la acción técnica dependerá de la métrica analizada, del problema identificado y del contexto particular de la aplicación.

Como síntesis de esta relación, la siguiente tabla presenta ejemplos de métricas asociadas con diferentes dimensiones de un sitio web, las herramientas que pueden utilizarse para su diagnóstico, valores de referencia que requieren atención y posibles acciones técnicas de optimización:

Tabla 5. Matriz de métricas y acciones de optimización

Métrica / evaluador	**Herramienta de diagnóstico**	**Criterio de referencia para la evaluación**	**Acción técnica de optimización**
Métrica LCP.	PageSpeed Insights o Lighthouse.	> 4,0 segundos.	Optimizar las dimensiones de las imágenes, convertirlas a formatos WebP o AVIF e implementar lazy loading.
Tasa de errores 5xx.	Registros del servidor Nginx.	> 1 % de las peticiones.	Ajustar el grupo de conexiones de la base de datos y optimizar consultas SQL.
Puntuación de accesibilidad.	axe DevTools.	< 90 puntos.	Agregar atributos aria-label faltantes y corregir la jerarquía de encabezados (h1-h6).
Tasa de abandono.	Google Analytics 4 o Matomo.	> 60 % en formulario.	Reducir el número de campos obligatorios e implementar

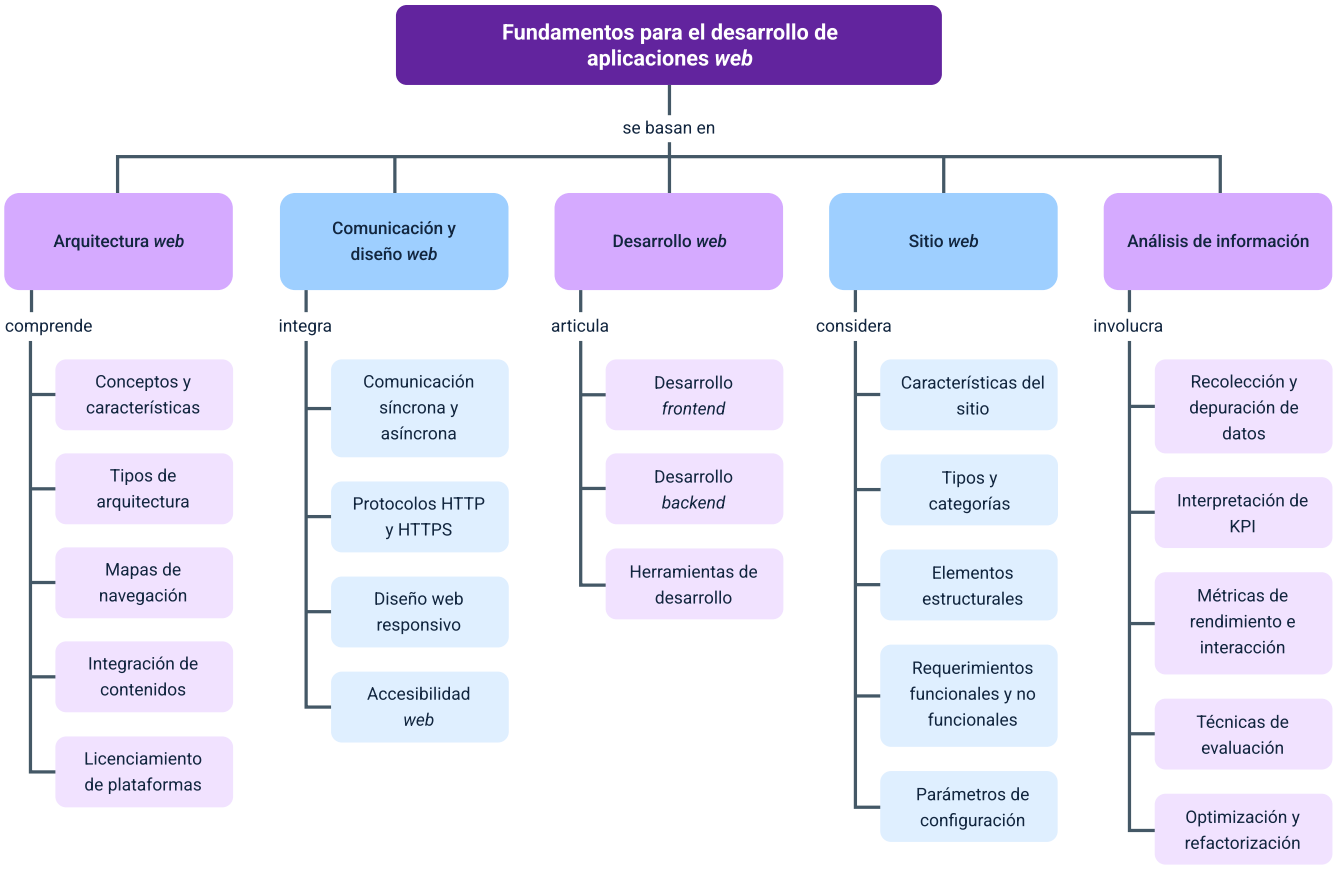
Métrica / evaluador	Herramienta de diagnóstico	Criterio de referencia para la evaluación	Acción técnica de optimización
			validación instantánea en el cliente.

Nota: los valores presentados constituyen criterios de referencia para orientar la evaluación y deben interpretarse de acuerdo con los objetivos, requerimientos y contexto de cada sitio o aplicación web.

Síntesis

La construcción de sitios web bajo criterios de interoperabilidad y protocolos técnicos requiere integrar la infraestructura tecnológica, el diseño orientado a la usabilidad y el análisis continuo de información.

El proceso inicia con una arquitectura que define la estructura y el licenciamiento del sistema. Posteriormente, se abordan los protocolos de comunicación y las técnicas de diseño responsivo, procurando que la interfaz responda a criterios de accesibilidad y rendimiento. El desarrollo requiere el uso de herramientas de edición, construcción y control de versiones. Una vez publicado el sitio web bajo los parámetros de configuración correspondientes, el análisis de información permite recopilar e interpretar métricas que orientan la identificación de oportunidades de mejora y favorecen un ciclo continuo de optimización técnica y funcional.


Fundamentos para el desarrollo de aplicaciones *web*
se basan en
Arquitectura *web*
comprende
Conceptos y características
Tipos de arquitectura
Mapas de navegación
Integración de contenidos
Licenciamiento de plataformas
Comunicación y diseño *web*
integra
Comunicación síncrona y asíncrona
Protocolos HTTP y HTTPS
Diseño web responsivo
Accesibilidad *web*
Desarrollo *web*
articula
Desarrollo *frontend*
Desarrollo *backend*
Herramientas de desarrollo
Sitio *web*
considera
Características del sitio
Tipos y categorías
Elementos estructurales
Requerimientos funcionales y no funcionales
Parámetros de configuración
Análisis de información
involucra
Recolección y depuración de datos
Interpretación de KPI
Métricas de rendimiento e interacción
Técnicas de evaluación
Optimización y refactorización

Glosario

API (Application Programming Interface): conjunto de definiciones y mecanismos que permiten la comunicación e integración entre diferentes aplicaciones o componentes de software.

DNS (Domain Name System): sistema que permite asociar nombres de dominio con las direcciones necesarias para localizar recursos y servicios en una red.

DOM (Document Object Model): modelo que representa la estructura de un documento HTML y permite acceder y modificar sus elementos, contenido y propiedades mediante programación.

Framework: estructura de desarrollo que proporciona componentes, reglas y herramientas reutilizables para facilitar la construcción y organización de aplicaciones.

JSON (JavaScript Object Notation): formato ligero de intercambio de datos utilizado para estructurar y transmitir información entre diferentes componentes o sistemas.

KPI (Key Performance Indicator): indicador clave de rendimiento utilizado para medir y analizar el comportamiento de un proceso, sistema o aplicación respecto de objetivos definidos.

Refactorización: proceso de reorganización y mejora de la estructura interna del código sin modificar su comportamiento funcional esperado.

SLA (Service Level Agreement): acuerdo que establece niveles y condiciones de servicio esperados, como disponibilidad, rendimiento o tiempos de respuesta.

TLS (Transport Layer Security): protocolo criptográfico utilizado para proteger la comunicación mediante mecanismos de cifrado, integridad y autenticación.

Wireframe: representación esquemática de una interfaz que permite definir la distribución y jerarquía de sus principales elementos antes de su desarrollo visual.

Referencias bibliográficas

Chacon, S. & Straub, B. (2014). Pro Git (2.ª ed.). Apress.

Docker, Inc. (s. f.). Docker Documentation: Get started. Docker Documentation.

ECMA International. (2023). ECMA-262: ECMAScript® 2023 Language Specification (14.ª ed.). ECMA International.

Fielding, R., Nottingham, M. & Reschke, J. (2022). HTTP Semantics (RFC 9110). Internet Engineering Task Force.

MDN Web Docs. (s. f.). Web Developer Guides. Mozilla.

NGINX. (s. f.). Beginner's Guide. NGINX Documentation.

World Wide Web Consortium (W3C). (2025). Web Content Accessibility Guidelines (WCAG) 2.1. W3C.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema Virtual de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Experto temático	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
Oscar Ivan Uribe Ortiz	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Manuel Felipe Echavarria Orozco	Desarrollador `full stack`	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima
María Fernanda Pineda Mora	Evaluadora de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculador de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima